Competitive Analysis of Caching With Delayed Hits on Line Networks

Saatvik Kambhampati
Department of Computer Science
University of Warwick, Coventry, United Kingdom
Saatvik.Kambhampati@warwick.ac.uk

Florin Ciucu (Supervisor)
Department of Computer Science
University of Warwick, Coventry, United Kingdom
F.Ciucu@warwick.ac.uk

August 19, 2026

Abstract

Competitive analyses of caching with delayed hits (DHs) have so far been focused on single-cache systems, where a cache-miss costs a fixed constant and a cache-hit costs nothing. In industry, caches are often implemented as networks where the cost of serving a request scales with the distance it travels upstream, so hits are no longer zero-latency. This report formalises a so-called 'DH-aware' line-network model, characterised by the number of caches $|V|$, the per-cache capacity k, and the inter-cache delay Z_A. It further establishes two lower-bounds on the competitive ratio of deterministic online policies. Applying existing adversary sequences yields a trivial $\Omega(1)$, as it does not grow with network-specific parameters. This adversary is shown to be indifferent to the hit-cost that networks introduce, treating the network as a single cache. Such a result motivated the remaining work: a second adversary, tailored to the Leave-a-Copy-Everywhere (LCE) placement policy, yields $\Omega(k|V|)$. This is the first competitive bound for LCE content placement policy on a cache network, and it provides a bound that grows with the length of the network $|V|$, and the size of each cache k. Remaining work targets a matching upper-bound and simulation-based validation.

Contents

1 Introduction **3**

2 Problem & Objectives **3**
2.1 Objectives 4

3 Background Research **5**
3.1 Classical Caching & Caching With Delayed Hits 5
3.2 Eviction Policies & Content Placement Policies 5
3.3 Competitive Analysis 6
3.3.1 Upper-Bound Analysis Techniques 7

4 Caching Model **7**
4.1 Formulation of Caching & Delayed-Hits 7
4.1.1 TTL Eviction Policy 10
4.1.2 Assumptions 10
4.2 Cache Networks 10
4.2.1 The LCE Placement Policy 12
4.2.2 Cache Line Networks 12
4.2.3 Assumptions for Cache (Line) Networks 13
4.3 Competitive Analysis for Caching 13
4.4 Finding the Lower-Bound 15
4.5 Finding the Upper-Bound 15

5 Competitive Analysis of LCE on Line Networks **15**
5.1 Choosing $\mathcal{O}$ For CPPs 16
5.2 Initial Lower-Bound 16
5.3 Lower-Bounding LCE 19
5.3.1 Adversarial Sequence for LCE 19
5.3.2 LCE Lower-Bound 21
5.4 Why the Gurushankar Upper-Bound Proof Does not Hold on Networks 23
5.4.1 The Denominator 23
5.4.2 What Does Not Hold: Non-Zero Latency Hits of $\mathcal{P}$ 23
5.4.3 Why $t' \geq t_2$ hits Cannot Be Bounded by Marking 24

6 Simulation Results **24**
6.1 Simulation of ρ_{MW} on Line Networks 24
6.2 Simulation of ρ_A on Line Networks 24
6.3 Simulation of 24

1 Introduction

Caching's importance, as a technique to improve and contribute to the quality of service of networks and systems, has become increasingly recognised [1]. Geographically distributed, intermediate data servers such as Content Delivery Network (CDNs) act as a 'middleman' for distributing data, which originates from a central server, to end users [2]. They provide low-latency access to frequently-requested data that may otherwise take far longer for a user to retrieve [3]. In general, such a concept is known as a *cache* and has, traditionally, been deployed across CPU hierarchies in hardware, and as CDNs in networked systems [2, 3].

In caching, a client requests data from the cache first: if the item is present, the cache serves it directly (a 'cache-hit'); otherwise the request is forwarded to a *backing-store* (a 'cache-miss') [3], containing all possible items, which returns the item back to the client. Whether a fetched item is then admitted to the cache, and which item, if any, is evicted from the cache to make room, is decided by an *eviction policy* (sometimes called a *cache replacement policy*) [4].

The time between a request arriving and the backing-store returning a fetch for that item (assuming the request resulted in a cache-miss) is known as the *fetch time*. From this, a specific property of caching models may be defined known as the *delay parameter*. This is the ratio of the fetch time to the inter-request arrival time[1] [5]. This property essentially defines the number of requests that arrive at the cache while a fetch request for that same item is already en-route.

In classical caching, this delay parameter is set as zero (implying that the fetch time is zero). However, real systems may have a tangible, non-negligible lag between the cache and the backing store, such that the fetch time is not zero. This has been observed in high-throughput systems, such as in FPGAs [5], and also in CDN hierarchies [3]. Suppose that two requests for the same item are to arrive at a cache, which does not have this item. The first request results in a cache-miss, and a fetch request is sent to the backing-store. Then, the second request arrives while this response is still in-flight. In a classical model, this results in a cache-hit as the item is instantly returned from the backing-store and admitted to the cache. However, in non-classical models, the request is placed into a queue. When the fetch for the first request is resolved, the second request is taken out of a queue and is considered a *delayed-hit* (DH). Models in which DHs are accounted for are identified as 'DH-aware' contexts or models, with the converse being 'DH-unaware' contexts.

The behaviour and process for caching works slightly differently in *cache networks* (such as CDNs), compared to so-called 'single-cache systems'. A cache network is, informally, a group of inter-connected caches that can forward requests to one-another (if they cannot serve the request themselves). The network, ultimately, is connected to one backing-store, and may have one or more 'ingress' caches (caches to which requests are initially sent). Such a network allows for more data to be stored, over a larger physical distance. This means that they can serve geographically dispersed clients with lower latency. A cache serving a client in one area may store data that is relevant to that area specifically, and may query more distant caches for data that is less frequently requested.

2 Problem & Objectives

Much of the current caching literature only analyses single-cache systems with delayed-hits *or* caching networks under non-DH conditions. There is little literature regarding competitive analyses of cache networks *with* delayed-hits, which is the gap that this project aims to contribute to. It aims

[1] The inter-request arrival time is the time between two requests for the same item

to extend the competitive analysis of DH-aware caching to the line-network topology, from single-cache systems. Specifically, the behaviour of the Leave-a-Copy-Everywhere (LCE) content placement policy (CPP)[2] considered on a line network in a DH-aware context. In LCE, when the backing-store returns a fetch, the item is admitted to every cache along the queried path. In this way, LCE introduces a high amount of redundancy [6]. Despite this, LCE specifically is studied here as it is heavily applied in literature, as a baseline policy [6, 7], and in industry [8]. Thus, it is the best policy on which to perform an initial competitive analysis. The central contribution of researching this specific gap is to provide a tight competitive analysis of LCE on DH-aware caching networks.

In DH-unaware contexts, a cache hit incurs zero-cost, regardless of from where the item is fetched [5]. DH-aware contexts fundamentally change this. If a cache fires a miss and forwards a fetch request, and the next cache in the network fires a hit, the delay incurred is that between two adjacent caches. This is less than the latency of forwarding to a deeper node, or to the backing-store. Thus, hit cost is now proportional to the 'fetch distance'. This provides an interesting problem to be solved in the context of competitive analysis as to the tightness guarantees of LCE on a network topology.



Figure 1: A 'single-cache system'.

2.1 Objectives

The objectives for this project are as follows:

1. Formally define the DH-aware line caching model, including the cost function and the LCE placement policy;
2. Construct an adversarial request sequence establishing a lower-bound;
   (a) Establish a baseline lower-bound using Manohar & Williams's adversarial sequence [9], which will help guide future work;
   (b) Establish the lower-bound for LCE;
3. Prove an upper bound on the competitive ratio (CR)[3] of LCE on a line of caches as a function of the number of caches, size of each one, and the delay parameter, thereby showing tightness;
4. Develop a DH-aware cache network simulator capable of modelling line topologies under configurable policies, cache sizes, and delay parameters;
5. Validate the theoretical bounds experimentally by comparing simulated CRs against the proven guarantees; and

[2] A CPP is a policy that determines *to which cache* a specific item should be admitted, in contrast to an eviction policy which determines *what* should be evicted from a specific cache

[3] The CR is a metric to evaluate the performance of algorithms in comparison to their optimum counterparts, and is introduced in more detail in Section 4.3

6. Investigate generalisations to tree topologies as extensions.

This report discusses objectives 1, 2(a), and 2(b); Section **??** outlines the plan for objectives 3–6.

3 Background Research

3.1 Classical Caching & Caching With Delayed Hits

The concept of caching can trace its origins, via 'paging', back to Kilburn et al.'s seminal 'One-Level Storage System' paper, published in 192 [10]. This system discussed a novel memory storage technique that was used in the Atlas computer. They originally proposed the idea of the 'core-drum store' system, which would appear as a single level of storage to the user. However, internally, the system would allow the core to query the drum for additional data. **WRITE MORE HERE**. This technique eventually became known as 'paging'.

Following this, in 1966, L.A. Belády proposed his optimal, offline eviction policy Min [11]. He originally proposed that no realistic eviction ('replacement') algorithm/policy can actually be the 'optimal' one because none of them are able to take into account future requests. Thus, the most optimal algorithm should be based on such information being available. The Min algorithm should read the entire input sequence, and then work backwards to build the 'minimum replacement sequence' [11]. The algorithm itself is fairly straight forward: Evict the item which will not be requested for the *longest* time into the future. This has been the standard optimum, offline algorithm for evictions used in *classical* caching, since its publication

Then, in 2020, Atre et al. [5] highlighted the effect that DHs could have on caching systems. They show that, by assuming the existence of a non-negligible fetch time, a request for an item that arrives during an existing fetch request for that item should not be classified as a cache-miss or a cache-hit. Atre et al. argues that neither is 'correct'. Instead, they propose a third classification: the delayed-hit. They argue that the cost incurred by this second request is in fact less than that of the first (because the second request 'waits' less time for the first request to be fulfilled). Thus, they classify it as a *delayed hit*. They show the sub-optimality of Belády's Min [11], when observed in a DH-aware context. Furthermore, they discuss the discrepancy between hit ratio and latency, whilst providing an alternative DH-aware metric: the Aggregated Delay. They then propose a new latency-optimal offline algorithm known as Belatedly. Finally, they introduce a novel online algorithm for DH-aware caching known as Minimum-AggregateDelay.

3.2 Eviction Policies & Content Placement Policies

Since almost the introduction of caching and paging, certain policies and algorithms have been designed that act as decision rules, to determine whether specific items are to be evicted or admitted to the cache. Further, in cache networks, there exist content placement policies that dictate *where* an item will be placed in the cache. All three of these have been extensively studied, however eviction policies remain the focus of much caching literature.

Popular eviction policies include LeastRecentlyUsed (LRU) and LeastFrequentlyUsed (LFU), both of which have been combined to form the LeastRecently-FrequentlyUsed (LRFU) policy by Li et al. in 2012 [12]. In all three policies, a number is associated with every item in the cache. In LRU, this number represents the time at which the item was last requested (and, upon eviction, the item with the time furthest in the past is evicted). Whereas, in LFU, this

number is a counter of how many times the item has been requested since it was last admitted to the cache (and the item with the lowest frequency is evicted). Li et al. then propose the LRFU policy for caching routers in 2012, combining the advantages of both policies. They do this by introducing a **C**ombined **R**ecency and **F**requency value (CRF) by using a function $f(x) = e^{-\gamma x}$, where $\gamma \in [0, 1]$ represents a weighting that allocates greater importance to either the *recency* or *frequency* property. They provide the computation of the CRF value for an object i at time t_i as

$$C_{t_i} = \text{CRF}(t_i) = \begin{cases} f(0) + f(t_i - t'_i)C_{t'_i} & , a(i) = 1 \\ f(0) & , a(i) = 0 \end{cases}$$

The Time-To-Live (TTL) eviction policy is also a staple of caching. TTL works by, canonically, evicting any items whose associated timer (counting down from their time-to-live value) reaches zero. They have further been noted for their relationship with and mapping to capacity-driven policies, such as LRU [13].

Separately, placement strategies for caching networks have been studied in DH-unaware contexts. Strategies such as LCE and Leave-a-Copy-Down (LCD) move content toward the client on cache hits [7]. An alternative approach, Demote [8, 14], instead pushes evicted items one hop upstream in the cache hierarchy on replacement, preserving recently-evicted content deeper in the network. While LCE is very prevalent in the literature, it has not been analysed under the DHs model or subjected to competitive analysis.

3.3 Competitive Analysis

Competitive Analysis is a method, first pioneered by Sleator & Trajan in 1985, that attempts to describe the worst-case performance of an *on-line* algorithm relative its optimal, *off-line* counterpart. It involves, centrally, the computation of the *competitive ratio*, which is the ratio of the 'cost' incurred by the online algorithm to the cost incurred by the optimal, offline algorithm, given an input sequence. They proved that the LRU eviction policy is $O(k)$-competitive (where k is the cache capacity). The optimal offline policy that they used was Belády's Min algorithm [11]. The competitive ratio upper- and lower-bounds were later defined again by Karlin et al. [15] in 1988, who originally coined the term *Competitive Analysis*.

In May 2020, Manohar and Williams [9] published a paper that proposed a competitive ratio lower-bound for any deterministic DH-aware online caching algorithm. Specifically, they find that the lower-bound grows with both the capacity of the cache and the delay parameter. In the same publication, they further prove that Atre et al.'s DH model did not exhibit anti-monotonicity and provide a relevant modification to their work and the model. Anti-monotonicity is a valuable property to retain in caching as, intuitively, a cache hit should result in a lower latency than a cache miss. Their modification is to simply fetch an item from the backing store, *even* when the request results in a cache-hit, resulting in a model that never 'rewards' a miss more than a hit.

In 2025, Gurushankar et al. [16] prove that the Least-Recently-Used (LRU) eviction policy, in a DH-aware context, has a CR upper-bound that scales with the cache capacity and *also* with the delay parameter. The two bounds 'match', meaning that this provides the first 'tight guarantee' for delayed-hits caching with the LRU. They prove that this guarantee extended to any marking algorithm. That year, Tan et al. [17] published a paper that shows any randomised caching algorithm has a CR lower-bound that scales with the cache capacity and the logarithm of the delay parameter ($\Omega(k \log Z)$). They then provide the outline for a novel randomised caching algorithm 'CaLa', proving the upper-bound to be $O(k \log Z)$-competitive, thereby matching the bounds and providing a tight guarantee.

3.3.1 Upper-Bound Analysis Techniques

Upper-bound analysis can be achieved through a variety of methods and techniques. Initially, Sleator & Trajan introduced the phase-partitioning technique, designed specifically for marking algorithms [18]. In such a technique, the input sequence can be partitioned into distinct phases, determined by the capacity of the cache. Then, by making an argument about how many misses there are at most or at least per-phase, the upper-bound of the system in general can be determined. This was then extended more formally in 1991 by Fiat et al., who showed that randomised online algorithms were H_k-competitive via phase partitioning [19]. Gurushankar et al.'s novel introduction of superphase partitioning stems directly from this [16].

The other main branch of upper-bound analysis stems from Buchbinder and Naor's 2009 publication [20] that introduced the idea of using Primal-Dual linear programs for competitive analysis, and its capabilities in aiding the design and analysis of online algorithms. They showed that the Dual LP acts as a lower-bound for the Primal LP, which can be used to derive online algorithm, and aid in identifying an upper-bound. Initially, they used the Primal-Dual approach to solve the *weighted paging problem*, in which every page (item) has a fetch-cost associated with it, that is charged on cache-misses. By redefining this cost on *evictions* instead (which they then 'bind' to misses), they provide a Primal-Dual program to eventually provide an $O(\log k)$-competitive 'fractional' algorithm. Then, Bansal et al. further showed that a randomised algorithm in the weighted paging problem was also $O(\log k)$-competitive via the Primal-Dual approach.

At around the same time, Bansal et al. (*) provided an $O(\log^2 k)$-competitive algorithm for the *generalised caching problem* – which is where each item in the universe has its own arbitrary weight/cost. Adamaszek et al. would later improve on this by introducing a new $O(\log k)$-competitive algorithm for generalised caching [21]. Both papers relied upon the Primal-Dual approach for this problem. They further reported that this improved bound unifies all edge-cases of the generalised caching problem, and matched the lower-bound, thereby providing tightness.

4 Caching Model

4.1 Formulation of Caching & Delayed-Hits

Below, each part of the caching pipeline and model in a classical caching context is defined. First, a sequence of requests is defined, along with a cache and its specific state. This pipeline is then shown visually in **Figure 2**.

Definition 4.1 (Sequence of Requests): Given a set of all items $\mathcal{I}$ and a set of time $T \subset \mathbb{R}_{\geq 0}$, let a request r_j be modelled as a tuple s.th. $r_j = (i, t)$ for $i \in \mathcal{I}, t \in T$. Then

$$\rho = (r_j)_{j=1}^{N}$$

is a sequence of requests.

Definition 4.2 (Cache and Cache State): Given a set of all items $\mathcal{I}$ and times $T \subset \mathbb{R}_{\geq 0}$, the state of a cache at a given time t is defined as a set

$$C_t = \{(i, \tau) \mid i \in \mathcal{I}, \tau \in T\} \subset \mathcal{I} \times T$$

Where τ represents some arbitrary and context-dependent time value that is associated with item i in the cache at state C_t. For example, in Time-to-Live policy caching, τ may represent the time-to-live of $(i, \tau) \in C_t$.
Hereinafter, a cache in general may be referred to as simply C, rather than its specific state at time t (C_t).

As requests to the cache from the sequence arrive, the cache determines if the item being requested triggers a *hit* or a *miss*:

Definition 4.3 (Hits and Misses): Given a time $t = t_1 \in T$, the request at t is $r_j = (i, t_1)$ and the cache state is C_{t_1}. Then, a *hit indicator* function $h : \mathcal{I} \to \{1, 0\}$ is defined as:

$$h(i) = \begin{cases} 1, i \in \{i' \mid (i', \tau) \in C_t\} \\ 0, i \notin \{i' \mid (i', \tau) \in C_t\} \end{cases}$$

, where $h(i) = 1$ means *hit*, and $h(i) = 0$ means *miss*.

Definition 4.4 (Fetch Times - [5]): At some time $t = t_1$, given the arrival function $h(i)$ in **Definition 4.3** and a backing store $B \subset \mathcal{I}$, if $h(i) = 0$ (i.e. a miss is fired), C triggers a *fetch* request of item i from B. B then returns i to C at time $t = t_b$. Then, the *retrieval time* of this operation is $F = t_b - t_1$.
By letting the *inter-request arrival time* be R, the time to fetch can be expressed as the ratio of retrieval time to inter-request arrival time:

$$Z = \frac{F}{R}$$

After a hit or miss is triggered by C, the specific caching policy is triggered to update the cache state:

In classical ('DH-unaware') caching, it is assumed that $Z \leq 1$, which means that the retrieval time is assumed to be essentially instantaneous. However, in DH-aware caching models, $Z > 1$.

Algorithm 1: UPDATE

Data: Cache state C_t, known as C, an incoming request $r_j = (i, t)$, and an eviction candidate $(i_{\text{evict}}, \tau_{\text{evict}})$.

Result: $C_{t'}$

if $h(i) = 0$ **then**

  Wait Z time-steps for B to return item i.

  $(i_{\text{evict}}, \tau_{\text{evict}}) \leftarrow \mathcal{P}(C_t)$ `/* Use the eviction policy to find a victim to evict. */`

  $C_{t'} \leftarrow (C_t \setminus \{(i_{\text{evict}}, \tau_{\text{evict}})\}) \cup \{(i, \tau)\}$ `/*` $t' = t + Z$ `here. */`

end

else if $h(i) = 1$ **then**

  $C_{t'} \leftarrow (C_t \setminus \{(i, \tau')\}) \cup \{(i, \tau)\}$ `/*` $t' = t + 1$ `here. Since the item hit, no need to evict anything. Simply update or reset` τ`, and only if needed. */`

end

return $C_{t'}$

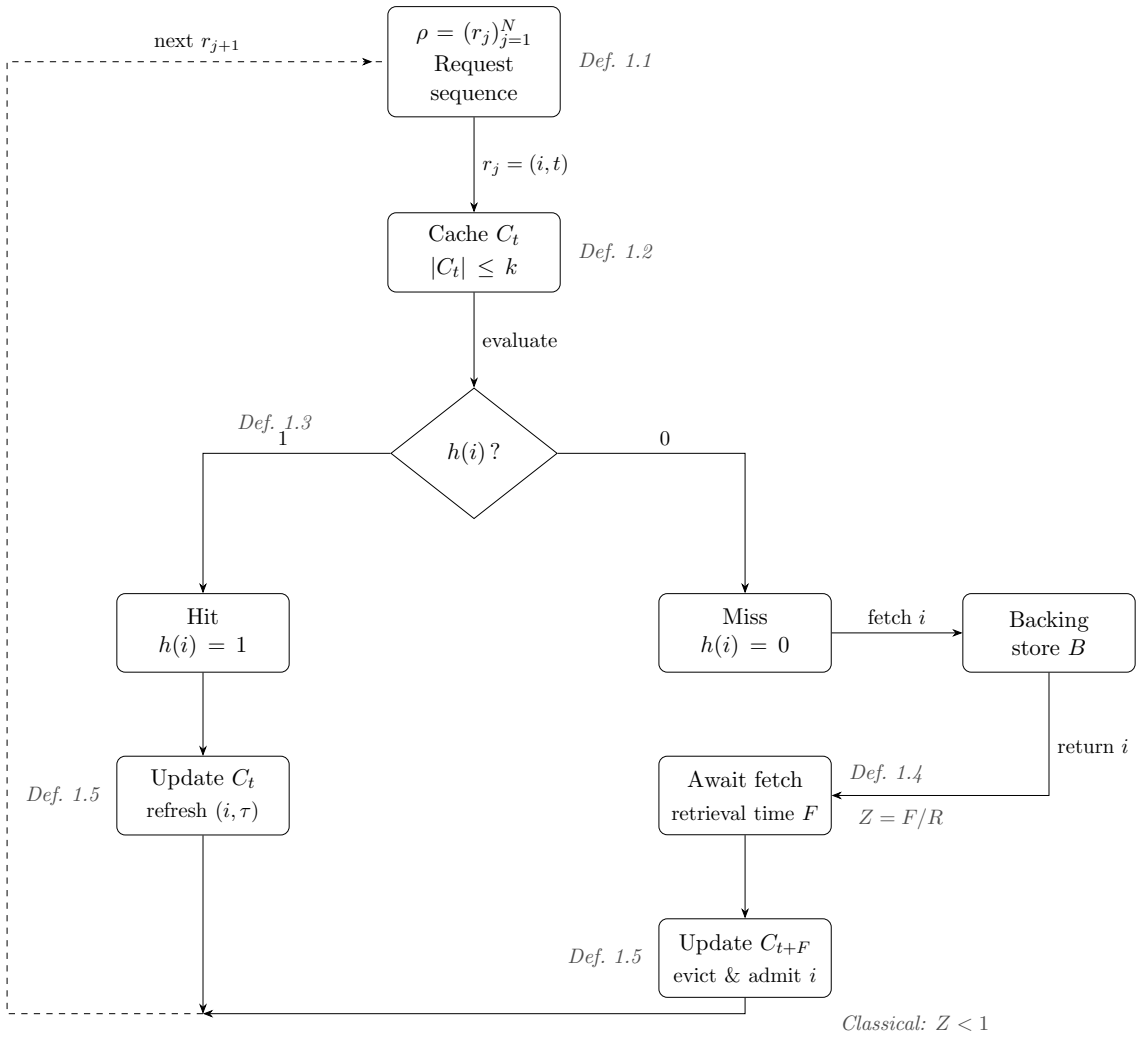


Figure 2: The classical caching pipeline.

The advent of DH-aware models means that the time between a request arriving and being fulfilled (the *latency*) becomes a tangible, meaningful metric through which a caching system can be measured. Manohar & Williams defined latency as follows:

Definition 4.5 (Latency): Given a single-cache system using a policy $\mathcal{P}$ and a sequence of requests $\rho = (r_1, r_2, r_3, \ldots, r_N)$, the latency of each request l_t is defined as:

$$l_t = (1 - h(i))(Z - (t - p_t)),$$

where p_t is the last request for the same item i requested at time t that resulted in a *cache-miss*.

4.1.1 TTL Eviction Policy

Time-**T**o-**L**ive is a specific type of eviction policy in which every item in the cache is paired with a counter (the 'TTL'). At every time-step, this counter counts down to zero. Two variants of TTL are standard in caching. In the first, the item is evicted *as soon as* the TTL is zero. In the second, items are only evicted when a new item must be admitted to the cache – at which point the policy chooses to evict the item with the lowest TTL. In this project, the latter variant was used. The analysis provided herein holds for all TTL values.

Algorithm 2: TIME-TO-LIVE

Data: The cache state at time $t + Z$, C_{t+Z}, known as C.
Result: The tuple $(i_{\text{evict}}, \tau_{\text{evict}}) \in C_{t+Z}$ that should be evicted.
victim $\leftarrow (\perp, \perp)$; `/* The (current) eviction candidate / victim */`
for $p \in C$ **do**
  $C^{(s)}_{t+Z_B+1} \leftarrow \text{update}(r_j, 0, C^{(s)}_{t+Z_B})$
end

FINISH ALGORITHM + TALK ABOUT TLL & MARKING

4.1.2 Assumptions

This section outlines some assumptions that are consistently used throughout this project.

Assumption 4.1 (Single R): The inter-request arrival time is a single scalar $R > 0$. That is, arrivals are both regularly spaced and item-independent. This idealisation follows the competitive-analysis convention of [5, 9].

Assumption 4.2 (Discrete times T): Times T are assumed discrete time-steps, i.e. $T \subseteq \mathbb{N}_0$. This is standard convention in caching competitive analysis [5, 9, 16].

4.2 Cache Networks

Here, the single-cache model is expanded upon into a cache network. Informally, this is a series of inter-connected caches and a backing store. Each network will have at least one 'ingress' or 'leaf'

cache to which requests are sent to, and there should be a path from every cache in the network to the backing-store. Definition 4.6 provides the *general* definition of a cache network, however this is later constrained to a *cache line network* for the remainder of this project.

Definition 4.6 (Cache Networks): Let $\mathcal{N}$ be a graph s.th. $\mathcal{N} = (V, E)$, where V is the set of vertices in the graph and $E \subseteq V \times V$ is the set of edges between two vertices in the graph. Then, $\mathcal{N}$ is a 'cache network' when:

1. Every vertex $v_x \in V$ represents a single cache $C_t^{(x)}$ (as in Definition 4.2);
2. There is a backing-store B connected to at least one cache vertex in the network;
3. There exists a valid path from any vertex to B; and
4. The network has at least one designated 'ingress' vertex (also referred to as a 'leaf') to which requests are initially sent.

Further, the total capacity of a cache network is $k_T = |V|k$, where k is the capacity of each cache (Definition 4.2). Finally, let $N \subset \mathcal{I}$ be the set of all items stored in the cache network $\mathcal{N}$ at any given time.

Ordinarily, when a request $r = (i, t)$ arrives at the leaf vertex v_0, if $h_0(i) = 0$ (note here $h_x(i)$ is the hit-indicator function (Definition 4.3) for cache $C_t^{(0)}$), then r is forwarded upstream to another cache-vertex v_x [13]. In this way, a network can be thought of as being made of multiple cache-backing-store pairs, connected in series. This justifies the introduction of the *inter-cache delay parameter*, Z_A:

Definition 4.7 (Inter-Cache Delay Parameter): Given a cache network $\mathcal{N}$, as in Definition 4.6, the inter-cache delay parameter is the delay parameter between two directly connected cache-vertices. That is, $\forall (v_x, v_y) \in E$ the latency incurred by forwarding a request from v_x to v_y is Z_A (Definition 4.4). The total delay between the request's ingress vertex and B is $Z_B = Z_A + \cdots + Z_A = |P|Z_A$, where $|P|$ is the number of caches in the path from ingress to B.

An important modelling choice is whether to classify hits at caches other than the ingress as *misses* or *hits*. If an item is located at a non-ingress cache, then it still incurs some delay dZ_A, indicating that it should perhaps be classified as a miss. However, the item was still located *somewhere* in the network, indicating the converse. Both interpretations have merit, and neither changes the technical result or the analysis, however it is important to state which is used to avoid confusion. Here, they are classified as hits. Specifically, 'non-zero-latency-hits' (NZLHs):

Definition 4.8 (Non-Zero-Latency-Hits): For some request $r_j = (i, t)$, if i is found at some cache-vertex v_x s.th. v_x is *not* an ingress, then this is still classified as a *hit* (i.e. $h_x(i) = 1$). That is, a request is only a miss if it arrives at B. This may sometimes be referred to as a 'non-zero-latency-hit', as it still incurs a latency of dZ_A, where d is the number of times a request is forwarded upstream until either it hits or reaches B.

A cache network processes requests in the following method (**CITE THIS**):

1. Request r_k for item $i \in \mathcal{I}$ arrives at time t to cache-vertex v_0.

2. If $h(i) = 1$ (hit) then v_0 returns the item, incurring latency $l_k = 0$.

3. If $h(i) = 0$ (miss), then v_0 forwards the request upstream, incurring a latency of $l_k = Z_A$. If v_0 can forward the request to many caches, then a *deterministic* algorithm must choose to which cache-vertex the requested is forwarded.

4. Repeat from step 2. Each time, $l_k \leftarrow l_k + Z_A$ until B is reached or $h(i) = 1$, at which point $l_k = d \cdot Z_A$, where d is the length of the request's path in the network.

Cache-networks, as a whole, are governed by *content placement policies* (CPPs), also known as *decision policies* [8]. This is similar to how single-cache systems are governed by *eviction policies*. The CPP determines *which* cache a given item is admitted to [8]. Then, that cache's specific eviction policy is triggered to determine which item the cache will evict in order to admit i. Common CPPs include DEMOTE and LCE [8].

In the process provided above, if the request is eventually forwarded to B, B returns item i to the cache network. The eviction policy and CPP are applied to determine the redistribution of items across the caching network.

4.2.1 The LCE Placement Policy

Formally, the LCE policy copies the missed item (now returned by a fetch from B) to *every* cache that was queried.

Algorithm 3: LEAVE-A-COPY-EVERYWHERE

Data: The request $r_j = (i, t)$ that caused i to be fetched from B and a set $S \subseteq V$ of queried caches in $\mathcal{N}$
Result: N/A
for $s \in S$ **do**
  $C^{(s)}_{t+Z_B+1} \leftarrow \text{update}(r_j, 0, C^{(s)}_{t+Z_B})$
end

FINISH ALGORITHM

4.2.2 Cache Line Networks

This project specifically analyses *line networks* of caches, so it remains to formally define them as a subset of all cache networks.

Definition 4.9 (Cache Line Networks): A cache network $\mathcal{N}$, as in Definition 4.6, is a *line network* if:

1. Every vertex is connected to, at-most, two other vertices;
2. There is exactly one vertex that is connected to B, and this vertex is connected to exactly one other non-B vertex; and
3. There is exactly one ingress vertex (the 'leaf'). The leaf is connected to exactly one other vertex.

A line network $\mathcal{N}$ has an overall delay parameter of $Z_B = |V|Z_A$.

Figure 3 shows a general line network.

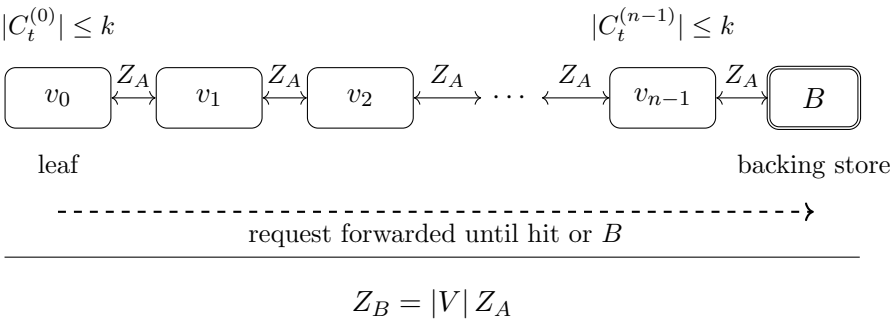


Figure 3: A line network of $|V|$ caches

4.2.3 Assumptions for Cache (Line) Networks

This section outlines assumptions made when analysing caching line networks in this project.

Assumption 4.3 (Homogenous Z_A): $\forall (v_x, v_y), (v'_x, v'_y) \in E$, each with an inter-cache delay of Z_A and Z'_A, respectively, for a given network $\mathcal{N}$ (Definition 4.6), $Z_A = Z'_A$.

Assumption 4.4 (Homogenous k): $\forall v_x \in V$, $|C^{(x)}| = k$

4.3 Competitive Analysis for Caching

With the caching, the line network, and latency now defined, the central metric of this project may be formalised: the *competitive ratio* (CR). This measures an online policy [4] against the optimal offline policy [5].

[4] An online policy is one that only knows past and current requests.

[5] An offline policy is one that can see the full request sequence, including future requests. Non-trivial offline policies are optimal ones.

Definition 4.10 (Competitive Ratio (Closed Form)): Given an online caching policy $\mathcal{P}$ and an offline optimal policy $\mathcal{O}$, the CR of $\mathcal{P}$ in a delayed-hits-aware context ($Z \geq 1$) is

$$\text{CR}(\mathcal{P}, \rho) = \frac{L(\mathcal{P}, \rho)}{L(\mathcal{O}, \rho)},$$

where $L(\mathcal{P}, \rho) = \sum_{r_j \in \rho} l_j$ (Def 4.1) is the total latency incurred by $\mathcal{P}$ on $\rho = (r_j)_{j=1}^{N}$ [16].

From Definition 4.10, it can be seen that it is ideal to obtain CR = 1 as this implies that $\mathcal{P}$ is as 'good' as $\mathcal{O}$ for the sequence ρ, with lower CRs being more optimal. However, this only describes the performance of $\mathcal{P}$ on a specific sequence ρ, not the performance of $\mathcal{P}$ *in general*. In order to describe it in general, the supremum over the sequence-space is used:

Definition 4.11 (Competitive Ratio): Given an online caching policy $\mathcal{P}$ and an offline optimal policy $\mathcal{O}$, the CR of $\mathcal{P}$ in a delayed-hits-aware context ($Z \geq 1$) is

$$\text{CR}(\mathcal{P}) = \sup_{\rho} \frac{L(\mathcal{P}, \rho)}{L(\mathcal{O}, \rho)},$$

where $L(\mathcal{P}, \rho) = \sum_{r_j \in \rho} l_j$ (Def 4.1) is the total latency incurred by $\mathcal{P}$ on $\rho = (r_j)_{j=1}^{N}$ [16].

Before continuing further, it is first important to understand fully what the CR shows: It is a description of the *worst-case* performance for $\mathcal{P}$. The lower-bound can then be defined as follows:

Theorem 4.1 (Competitive Ratio Lower-Bound): Let $\mathcal{P}$ be a deterministic online caching policy. Then $\exists \rho_A \in \mathcal{R}$ such that

$$CR(\mathcal{P}, \rho_A) \geq c \cdot x \implies CR(\mathcal{P}) \geq c \cdot x,$$

for some constant $c > 0$ and a variable parameter x. Equivalently,

$$\text{CR}(\mathcal{P}) = \Omega(x).$$

ρ_A is known as the 'adversarial sequence' [9, 15].

And the upper-bound is defined as:

Theorem 4.2 (Competitive Ratio Upper-Bound): Let $\mathcal{P}$ be a deterministic online caching policy. Then:

$$\forall \rho \in \mathcal{R},\ CR(\mathcal{P}, \rho) \leq c \cdot x',$$

for some constant $c > 0$ and a variable parameter x'. Equivalently,

$$\text{CR}(\mathcal{P}) = O(x').$$

Policy $\mathcal{P}$ is then referred to as '$O(x')$-competitive' [15, 16].

Finally, the analytic objective of this project is defined. When both bounds match, i.e. when

$x = x'$, it guarantees the competitive ratio. This is known as a *tightness guarantee*, and is the objective of the analysis.

In 2020, Manohar and Williams [9], proved the theoretical lower-bound for the CR of any deterministic policy acting in a DH-aware context:

> ***Theorem 4.3 (Manohar & Williams's Lower-Bound):*** In a single-cache system, given a deterministic online eviction policy $\mathcal{P}$ and an offline eviction policy $\mathcal{O}$, the competitive ratio of $\mathcal{P}$ has a lower-bound of
>
> $$\mathrm{CR}(\mathcal{P}) = \Omega(kZ),$$
>
> where k is the size of the cache, and Z is the delay parameter.

4.4 Finding the Lower-Bound

From Theorem 4.1, the method for finding the CR lower-bound can intuitively be extracted, and is further reinforced by Manohar & Williams' method [9].

1. Construct an adversarial request sequence ρ_A that maximises $\frac{L(\mathcal{P},\rho_A)}{L(\mathcal{O},\rho_A)}$
2. Compute $\frac{L(\mathcal{P},\rho_A)}{L(\mathcal{O},\rho_A)}$ in closed form
3. Determine the asymptotic lower bound on $CR(\mathcal{P})$.

4.5 Finding the Upper-Bound

The upper-bound presents a more time-consuming challenge than the lower-bound does, as it is a universal claim across the sequence-space. Thus, while the lower-bound can be approached by iterating the 'development' of a single sequence, the upper-bound must be proven by making claims that hold for all sequences. Gurushankar et. al. [16] claim that partitioning a general sequence into phases is standard in competitive caching analysis, and then they further partition those phases into 'super-phases'. Thus, beginning with phase partitioning may be a useful first, experimental step to take.

Finding the upper-bound consists of *upper*-bounding $L(\mathcal{P})$ (the numerator) and *lower*-bounding $L(\mathcal{O})$ and then computing the CR with these bounded values. It should, however, be noted that *this* lower-bounding is distinct from the general CR lower-bound. From Definition **??**, the CR lower-bound is defined by *one* adversarial sequence input, whereas the lower-bound denominator found for $L(\mathcal{O})$, by Definition **??**, must hold for *all* possible sequences.

5 Competitive Analysis of LCE on Line Networks

This section presents three main, novel contributions. The first is proof that the current lower-bound of $\Omega(kZ)$ does not hold for line networks with $|V|$ caches. The second is a placement policy-based lower-bound for line networks. Finally, a proposed upper-bound for placement policies on line networks is outlined. All three are preceded by introducing restrictions placed on $\mathcal{O}$, which are non-standard for CPPs.

5.1 Choosing $\mathcal{O}$ For CPPs

In literature, $\mathcal{O}$ represents the optimal, offline, *eviction* policy. These are Belády's MIN algorithm in classical caching [11], and the BELATEDLY algorithm for DH-aware caching [5]. It typically has the ability to evict whichever item it 'wants' to after each cache-miss. However, here, $\mathcal{O}$ represents the optimal, offline CPP [6]. It *cannot* determine which items are evicted from individual caches, as this is determined by the per-cache eviction policy. So, $\mathcal{O}$ should be able to freely place any item that B returns, and any item evicted by any cache, to any cache at its discretion.

Optimal eviction policies implicitly gain admission policy properties. They may admit an undesirable item and then evict it immediately, reverting the cache to its previous state. An optimal CPP, however, retains no such option because eviction is not under its control. Accordingly, $\mathcal{O}$ is granted the ability to decline placement outright.

5.2 Initial Lower-Bound

The Manohar–Williams sequence ρ_{MW} establishes $\Omega(kZ)$ on a single cache. Applying this sequence to a line network is the natural first step: it tests whether the existing adversary is topology-sensitive, and so provides a baseline that can guide future work.

First, initialise a cache line network $\mathcal{N}$ with the first k_T initial items in $\mathcal{I}$:

$$\begin{aligned} C_0^{(0)} &= \{i_1, i_2, \ldots, i_k\}, \\ C_0^{(1)} &= \{i_{k+1}, i_{k+2}, \ldots, i_{2k}\}, \\ &\ldots, \\ C_0^{(|V|-1)} &= \{i_{(|V|-1)k+1}, i_{(|V|-1)k+2}, \ldots, i_{k_T}\} \end{aligned}$$

Definition 5.1 (Bursty Request): Given a sequence of requests $\rho = (r_j)_{j=1}^{j=N}$ (Definition 4.1) and a delay parameter Z_B, a bursty request is a sub Q-tuple

$$\beta = (b_q)_{q=1}^{q=Q}$$

of ρ s.th. requests

- $b_1, \ldots, b_{Z_B}$ are non-requests (i.e. for $\perp$),
- $b_{Z_B+1}, \ldots, b_{2Z_B}$ are for some item $i \in \mathcal{I}$, and
- $b_{2Z_B}, \ldots, b_Q$ are again for non-requests (so $Q = 3Z_B$).

Further any request that is not for $\perp$ is referred to as an 'actual' request. Note that all actual requests in β are for the *same* item.

Definition 5.2 (Pure Request): Given a sequence of requests $\rho = (r_j)_{j=1}^{j=N}$ (Definition 4.1) and a delay parameter Z_B, a bursty request is a sub Q-tuple

$$\pi = (p_q)_{q=1}^{q=Q}$$

of ρ s.th. requests

- $b_1, \ldots, b_{Z_B}$ are non-requests (i.e. for $\perp$),
- b_{Z_B+1} is for some item $i \in \mathcal{I}$, and
- $b_{Z_B+2}, \ldots, b_Q$ are again for non-requests.

Next, define ρ_{MW} within the context of a *cache network*:

1. Make a pure request for $i \in \mathcal{I}$ s.th. $i \notin N$ (that is, i is not in any cache in the network).
2. Make a bursty request for $i \in \mathcal{I}$ s.th. $i \notin N$.
3. Repeat step 2 $k_T - 1$ more times.

The analysis for the lower-bound on a line network using ρ_{MW} first uses two lemmas before proving the main theorem (Theorem 5.1). Throughout, $\mathcal{P}$ is a deterministic, online placement policy, and $\mathcal{O}$ is the optimum, offline placement policy.

Lemma 5.1 ($\mathcal{P}$'s Latency): $\mathcal{P}$ incurs a latency of $L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}$.

Proof. The first request $r_1 = (i, 1)$ is a pure request (Definition 5.2). Because the requested item has been chosen s.th. $i \notin N \implies i \notin v_0 \wedge i \notin v_1 \implies h_0(i) = 0 \wedge h_1(i) = 0$. The request misses at both caches and so is classified as a cache-miss, and is fetched from B. Thus, $l_1 = Z_B$.

At $1 + Z_B$, when r_1's item is fetched, $\mathcal{P}$ may rearrange the network. Depending on the policy, item i may be rejected, or $\mathcal{P}$ may order one or all of the caches to evict an item. Regardless, after $t = 1 + Z_B$, there will be at least one item that is *not* in N.

Following r_1, there are k_T bursty requests. Each request in a burst is for the item not in N at the beginning of that burst. Let β_g represent the gth bursty request in ρ_{MW} (Definition 5.1). As stated previously, β_g begins with Z_B requests for $\perp$ (no item), followed by Z_B requests for some item i (an 'actual' request), followed by Z_B requests for $\perp$. If the first actual request misses, then the subsequent ones are delayed-hits. This is guaranteed as the adversary has chosen to request an item that is not in the network. Thus, requests $b_{Z_B+1}, \ldots, b_{2Z_B}$ incur a collective latency of $Z_B + (Z_B - 1) + \cdots + 1 = \frac{Z_B(Z_B+1)}{2}$ [9]. Finally, since there are k_T bursty requests, the 'bursty latency' of the sequence is

$$k_T \cdot \frac{Z_B(Z_B+1)}{2}$$

Giving:

$$L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}$$

□

Remark (Reparametrisation): This is precisely the result of Manohar & Williams ($L(\mathcal{P}, \rho_a) \geq Z + kZ\frac{Z+1}{2}$) [9], under the reparametrisation:

- $k \mapsto k_T$; and
- $Z \mapsto Z_B$.

Lemma 5.2 ($\mathcal{O}$ Hits on all Bursties): $\mathcal{O}$ hits on all bursty requests in ρ_{MW}.

Proof. The marking argument of Manohar & Williams [9] depends only on the cache holding a fixed number of distinct items; it is insensitive to whether that capacity is realised as one cache or as multiple (i.e. a network). Replacing the single-cache capacity k with the network capacity $k_T = |V|k$ therefore leaves the argument unchanged: after the initial fetch, $\mathcal{O}$ admits every requested item but the one unmarked $j \in \{1, \ldots, k_T + 1\} \setminus M$ and never rearranges again, so no bursty request is a full miss. Hence $\mathcal{O}$ hits on every bursty request. □

Lemma 5.3 ($\mathcal{O}$'s Latency): $\mathcal{O}$ incurs a latency of $L(\mathcal{O}, \rho_{\text{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$

Proof. Again, the first pure request $r_1 = (i, 1)$ incurs a miss for the same reason as in Lemma 5.1. This is because, although $\mathcal{O}$ knows what r_1 is requesting prior to the request arriving, it is unable to rearrange the network in response as it can only rearrange when B returns a fetch request. Thus, $l_1 = Z_B$.

Once again, at $t = 1 + Z_B$, $\mathcal{O}$ has the opportunity to rearrange the network as item i has been returned from v_0. From Lemma 5.2, the $\mathcal{O}$ line network hits on all bursty requests. The key difference here, though, is that such hits are not necessarily zero-latency.

Every hit incurs a latency of $d \cdot Z_A$ (Definition 4.8), where d is the number of times the request was forwarded upstream. From Lemma 5.2, after $t = 1 + Z_B$ (i.e. the return for the item requested in r_1), every item that will be requested in ρ_{MW} is already stored in N. Further, after r_1 there are k_T requests for distinct items,

$$\forall i \in N, \exists r_j \in \rho_{\text{MW}} \text{ s.th. } r_j = (i, t)$$

Let d be the number of forwards until a hit for a specific item $i \in N$. Then, for each bursty request β_g,

$$l_g = Z_B \cdot Z_A d.$$

Finding the total latency of all bursty requests is equivalent to summing $k(Z_B Z_A d)$ over the number of caches:

$$L_B = kZ_B Z_A \sum_{d=1}^{|V|-1} d = kZ_B Z_A \cdot \frac{(|V|-1)|V|}{2}$$

Substituting $Z_B = |V|Z_A$ (Definition 4.9):

$$L_B = kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

Finally:

$$L(\mathcal{O}, \rho_{\text{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

□

Theorem 5.1 (Initial Lower-Bound): In a cache line network $\mathcal{N}$ (Definition 4.9), given a deterministic online placement policy $\mathcal{P}$ and an offline eviction policy $\mathcal{O}$, the competitive ratio of $\mathcal{P}$ has a lower-bound of

$$\mathrm{CR}(\mathcal{P}) = \Omega(1).$$

Proof. From Lemma 5.1 and Lemma 5.3, respectively,

$$L(\mathcal{P}, \rho_{\mathrm{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2},$$
$$L(\mathcal{O}, \rho_{\mathrm{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

Then, from Definition 4.10:

$$\mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq \frac{Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}}{Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}}$$
$$\implies \mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq \frac{2 + k|V|^2 Z_A + k|V|}{2 + k|V|^2 Z_A - k|V|Z_A}$$

Then, as $|V|, k, Z_A \to \infty$:

$$\mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq \frac{k|V|^2 Z_A}{k|V|^2 Z_A - k|V|Z_A}$$
$$\mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq \frac{k|V|^2 Z_A}{k|V|Z_A(|V|-1)}$$
$$\mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq \frac{|V|}{|V|-1}$$
$$\mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq 1$$

Finally, using Theorem 4.1:

$$\mathrm{CR}(\mathcal{P}) \geq \mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq 1$$
$$\implies \mathrm{CR}(\mathcal{P}) = \Omega(1)$$

□

5.3 Lower-Bounding LCE

5.3.1 Adversarial Sequence for LCE

Before outlining the adversarial sequence that has been constructed in order to yield the required lower-bound, a set W must be defined. This will act as a useful tool to bound the sequence to ensure its finiteness.

Definition 5.3 (The Set W): Given a line network with $|V|$ caches, each of capacity k, let W be a set s.th.

$$W \subset \mathcal{I} \wedge |W| = k+1$$

Remark (Significance of W): By literature convention [5, 9, 16], a single cache system is initialised with the first k-distinct items of a size-ordered sequence of $\mathcal{I}$. Thus, exactly k items in W reside in $C^{(0)}$ of the line network, and one item at $C^{(1)}$.

Then, an example 'toy' network is outlined below which will be used to initially build the intuition for the sequence defined by Definition 5.4. This is known hereinafter as the $(2, 2)$-Network, or $\mathcal{N}_{(2,2)}$

Example 5.1 (The (2,2) Network): A line network with $|V| = 2$, $k = 2$, $Z_A = 2$ ($Z_B = 4$) and $\mathcal{I} = \{a, b, c, d, e, f\}$, $W = \{a, b, c\}$, TTL $= 6$, is used as a trace throughout this analysis. At $t = 0$, $v_0 = \{(a, 6), (b, 6)\}$, $v_1 = \{(c, 6), (d, 6)\}$.

Finally, the adversarial sequence is defined below.

Definition 5.4 (Adversarial Sequence ρ_A): Given a line network $\mathcal{N}$ with $|V|$ caches, each of capacity k and using the TTL eviction policy, and a universe of items $\mathcal{I}$, let ρ_A,

1. Make a pure requests for any item not in N at $t = 1$.
2. Wait / make $\perp$-requests for Z_B time steps.
3. Make a bursty request β_g for an item $i \in \mathcal{I}$ s.th. $i \in W \wedge i \notin N$ at $t = 5$.
4. Repeat step 2 for $|W| - 1$ more times.

be an adversarial sequence used on $\mathcal{N}$.

Remark (ρ_A on the $(2, 2)$-Network): On the $(2, 2)$-Network shown in Example 5.1, ρ_A reduces to:

$$\rho_A = (e/f, \beta_1, \beta_2, \beta_3)$$

, where β_1 requests item a, β_2 item c, and β_3 item b.

Because $|V|k < |\mathcal{I}|$ for any non-trivial cache network, there exists at least *one* $i \in \mathcal{I}$ s.th. $i \notin N$ at $t = 0$. Request this item at $t = 1$. Since $\mathcal{I} \setminus N = \{e, f\}$, requesting e or f results in a full miss and a fetch request (resolving at $t = 5$). Here, it would be beneficial to take advantage of DHs and continuously request e for 'free' latency. However, it inadvertently would disadvantage $\mathcal{O}$. Neither $\mathcal{O}$ or $\mathcal{P}$ can rearrange the network *before* the first fetch return. Thus, for now, any latency that $\mathcal{P}$ incurs will also be incurred by $\mathcal{O}$, which is counter-productive.

At $t = 5$ the fetch for e/f returns. The state of the network at this time is $C_4^{(0)} = \{(a, 1), (b, 1)\}, C_4^{(1)} = \{(c, 1), (d, 1)\}$. LCE mandates that e/f (take e from now on) is admitted at every cache, so the eviction policy evicts the item with the lowest TTL in each cache, thus providing $C_5^{(0)} = \{(e, 6), (b, 1)\}, v_1 = \{(e, 6), (d, 5)\}$. Now, each element of W is continuously requested (via bursty requests) in an order such that all bursty requests incur misses. This is possible as, after $t = 5$, there is always at least one item in W that is not in the network. Thus, for this trace, the final

adversarial sequence on $\mathcal{N}_{(2,2)}$ is:

$$\rho_A = (e/f, a^{Z_B}, c^{Z_B}, b^{Z_B})$$

5.3.2 LCE Lower-Bound

The analysis for the lower-bound on a line network using ρ_A first uses two lemmas before proving the main theorem (Theorem 5.2). Throughout, LCE is the deterministic LCE online placement policy, and $\mathcal{O}$ is the optimum, offline placement policy.

Lemma 5.4 (LCE's Latency): LCE incurs a latency of $L(\text{LCE}, \rho_{\text{MW}}) \geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2$.

Proof. As previously, r_1 incurs a miss as $k_T \leq |\mathcal{I}|$, so $l_1 = Z_B$.

Then, at $t = 5$, the item for r_1 has been fetched and the network admits it under LCE. Now all caches in $\mathcal{N}$ have this item, and have evicted one item each under TTL. Due to the way W has been constructed (Definition 5.3), there is at least one item $i \in W \wedge i \notin N$ which has also not yet been requested. The subsequent bursty request β_1 requests that item. As shown in Lemma **??**, bursty requests that incur all misses yields a latency of $\frac{Z_B(Z_B+1)}{2}$, and since there are $k+1$ bursty requests:

$$\begin{aligned} l_B &= (k+1) \cdot \frac{Z_B(Z_B+1)}{2} \\ &= \frac{Z_B(k+1)(Z_B+1)}{2} \\ &= \frac{kZ_B^2 + kZ_B + Z_B^2 + Z_B}{2} \end{aligned}$$

Giving:

$$\begin{aligned} L(\mathcal{P}, \rho_A) &\geq Z_B + l_B \\ L(\mathcal{P}, \rho_A) &\geq Z_B + \frac{1}{2}kZ_B^2 + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 + \frac{1}{2}Z_B \\ &\geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 \end{aligned}$$

□

Lemma 5.5 ($\mathcal{O}$ Hits on all Bursties): $\mathcal{O}$ hits on all bursty requests in ρ_A.

Proof. All bursty requests $\beta_1, \beta_2, \beta_3$ (Definition 5.4) are for items that are already in the cache at $t = 5$ (when r_1's item returns) by Definitions 5.4 and 5.3. Thus, all bursty requests will incur cache-hits. □

Remark (NZLHs on $\mathcal{O}$): However, although some will be NZLHs (Definition 4.8), $\mathcal{O}$ cannot do better than this by rearranging as there will always be exactly one item that is burstily-requested that is not in the leaf cache.

Lemma 5.6 ($\mathcal{O}$'s Latency): $\mathcal{O}$ incurs a latency of $L(\mathcal{O}, \rho_A) = Z_B + Z_B \cdot Z_A$

Proof. Like with LCE in Lemma 5.4, $\mathcal{O}$ has a full cache-miss with r_1 and thus incuiirs $l_1 = Z_B$.

From Lemma 5.5, $\mathcal{O}$ experiences all cache-hits on the $\beta_1, \beta_2, \beta_3$ bursty request sequences (Definition 5.4). It only remains to show the latency incurred by these hits, as some of them are, again, NZLHs.

Due to the way W is defined (Definition 5.3) and the way ρ_A is constructed (Definition 5.4), $\mathcal{O}$ has exactly k zero-latency hits and one NZLH. This one NZLH is guaranteed to be at vertex v_1's cache. Thus, the full bursty portion of ρ_A incurs $Z_B Z_A$ on ρ_A (as there are Z_B requests for the item at v_1's cache).

Putting this together yields:

$$\begin{aligned} L(\mathcal{O}, \rho_A) &= Z_B + l_B \\ &= Z_B + Z_B \cdot Z_A \end{aligned}$$

□

Theorem 5.2 (LCE Lower-Bound): In a cache line network $\mathcal{N}$ (Definition 4.9), given the LCE placement policy LCE and an offline eviction policy $\mathcal{O}$, the competitive ratio of LCE has a lower-bound of

$$\mathrm{CR}(\mathrm{LCE}) = \Omega(k|V|).$$

Proof. From Lemma 5.4 and Lemma 5.6, respectively,

$$\begin{aligned} L(\mathrm{LCE}, \rho_A) &\geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2, \\ L(\mathcal{O}, \rho_{\mathrm{MW}}) &= Z_B + Z_B \cdot Z_A \end{aligned}$$

Then, from Definition 4.10:

$$\begin{aligned} \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{\frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2}{Z_B + Z_B \cdot Z_A} \\ \implies \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{2 + k|V|^2 Z_A + k|V|}{2 + k|V|^2 Z_A - k|V|Z_A} \end{aligned}$$

Then, as $|V|, k, Z_A \to \infty$:

$$\begin{aligned} \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{\frac{1}{2}kZ_B^2}{Z_B \cdot Z_A} \\ \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{kZ_B^2}{2} \cdot \frac{1}{Z_B Z_A} = \frac{kZ_B}{2Z_A} \end{aligned}$$

Finally, substituting in $Z_B = |V|Z_A$:

$$\begin{aligned} CR(\mathrm{LCE}, \rho_A) &\geq \frac{k|V|Z_A}{2Z_A} \\ &\geq \frac{k|V|}{2} \end{aligned}$$

Finally, using Theorem 4.1:

$$\begin{aligned} \mathrm{CR(LCE)} &\geq \mathrm{CR(LCE}, \rho_A) \geq \frac{k|V|}{2} \\ \implies \mathrm{CR(LCE)} &= \Omega(k|V|) \end{aligned}$$

□

5.4 Why the Gurushankar Upper-Bound Proof Does not Hold on Networks

The method that Gurushankar et al. [16] used for their CR upper-bound depends upon phase and super-phase partitioning: the sequence is partitioned into 'phases', and the latency incurred within each phase is bounded using the marking property[6]. Most of their analysis and method holds for a line network, however one part of the argument does not. This section will outline what does and does not hold.

5.4.1 The Denominator

The lower bound on $L(\mathcal{O})$ is a pigeonhole argument. A super-phase (defined, informally, as a block of multiple phases) requests $k+1$ distinct items, so even the offline optimum must miss at least once within it. On a line network the same counting applies to the overall capacity k_T, and their Lemma 13 holds under the remapping $k \mapsto k_T$, $Z \mapsto Z_B$. So, knowing that this holds, it only remains to find the numerator, $L(\mathcal{P})$.

5.4.2 What Does Not Hold: Non-Zero Latency Hits of $\mathcal{P}$

The proof fails in a certain portion of the upper-bounding of $L(\mathcal{P})$. Their Lemma 10 bounds the per-phase latency by $kZ\min\{Z, |P^i|\}$. They prove this by first defining two values: t_1 (the first time an item is requested in a given phase), and t_2, defined as below:

$$t_2 = \begin{cases} t_1, & \texttt{Hit} \\ t_1 + Z, & \texttt{Miss} \\ t_1 + (Z - i(t_1)), & \texttt{Delayed-Hit} \end{cases}$$

The proof then splits into three distinct cases of t_2. If $t_2 = t_1$, then the item was already in the cache when it was initially requested at t_1 (i.e. it is already marked). If $t_2 = t_1 + Z$, then the item missed at t_1 and is now in the cache at t_2 (it is now marked). And the final case is the standard delayed-hit case. Thus, the interval (t_1, t_2) is the main focus of the original paper, as this is where potential delayed-hits will be found. Any request at time $t' \geq t_2$ is automatically a hit, due to marking, and is thus dismissed (as hits are always zero-latency in the single-cache model).

However, on a line-network, this implication is false. Hits are not always zero-latency on networks, as their incurred latency is proportional to their hit depth. Marking records cache residency; however it does not record location. Thus, the provided phase partitioning methods are unable to bound the latency of any hits that occur at times $t' \geq t_2$.

[6]Marking is a property of so-called 'marking algorithms' wherein, per phase, if a marking algorithm has 'seen' an item once, the item will not be evicted from the cache within that phase. The item is 'marked'. Per phase, only *unmarked* items may be evicted from the cache.

5.4.3 Why $t' \geq t_2$ hits Cannot Be Bounded by Marking

A cache system's state is defined by its contents at a particular time, and state transitions are defined as a 3-tuple (Item fetched, Item admitted, Item evicted) – i.e. the items that are fetched, admitted, and evicted at each transition. Marking works by constraining this state transition space by constraining, specifically, eviction. A marked item cannot be evicted, so the property immediately constrains the transition space by discarding the transitions that marking blocks. This constraining is the mechanism by which phase partitioning determines a latency bound.

A hit triggers *none* of the operations included state transition, thus marking cannot constrain the state transition space in accordance adequately. Furthermore, there are no operations that can be added to state transitions to provide such a constraint. For example, the only change that make to caches is that the TTL value is refreshed. This only changes the eviction order at the cache, but changes still require admission pressure. The transition therefore records nothing about the depth at which the request was served, and so deep hits, although non-zero-latency, result in no partition-able event.

6 Simulation Results

6.1 Simulation of ρ_{MW} on Line Networks

Showing that it's constant $\Omega(1)$

6.2 Simulation of ρ_A on Line Networks

Showing that it's $\Omega(k|V|)$

6.3 Simulation of

References

[1] B. Wu, W. Bao, and B. B. Zhou, "Competitive analysis of online elastic caching of transient data in multi-tiered content delivery network," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 12, pp. 2449–2462, 2024.

[2] Cloudflare, "What is an edge server?" https://www.cloudflare.com/en-gb/learning/cdn/glossary/edge-server/, n.d., accessed: 23 February 2026.

[3] B. Jiang and C. Ma, "Modeling and optimizing latency for delayed hit caching with stochastic miss latency," *arXiv preprint arXiv:2505.15531*, May 2025. [Online]. Available: https://arxiv.org/abs/2505.15531

[4] A. Jain, C. Lin, N. Jerger, and M. Martonosi, *Cache Replacement Policies*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2019. [Online]. Available: https://books.google.co.uk/books?id=QRieDwAAQBAJ

[5] N. Atre, J. Sherry, W. Wang, and D. S. Berger, "Caching with delayed hits," in *Proceedings of the ACM SIGCOMM*. ACM, 2020, pp. 495–510.

[6] S. Ioannidis and E. Yeh, " Adaptive Caching Networks With Optimality Guarantees ," *IEEE/ACM Transactions on Networking*, vol. 26, no. 02, pp. 737–750, Mar. 2018. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TNET.2018.2793581

[7] A. Ioannou and S. Weber, "A survey of caching policies and forwarding mechanisms in information-centric networking," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 4, pp. 2847–2886, 2016.

[8] G. Zhang, Y. Li, and T. Lin, "Caching in information centric networking: A survey," *Computer Networks*, vol. 57, no. 16, pp. 3128–3141, 2013, information Centric Networking. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128613002235

[9] P. Manohar and J. Williams, "Lower bounds for caching with delayed hits," *arXiv preprint arXiv:2006.00376*, 2020. [Online]. Available: https://arxiv.org/abs/2006.00376

[10] T. Kilburn, D. B. G. Edwards, M. J. Lanigan, and F. H. Sumner, "One-level storage system," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 2, pp. 223–235, 1962.

[11] L. A. Belady, "A study of replacement algorithms for virtual-storage computer," *IBM Syst. J.*, vol. 5, pp. 78–101, 1966. [Online]. Available: https://api.semanticscholar.org/CorpusID:8344798

[12] Z. Li, G. Simon, and A. Gravey, "Caching policies for in-network caching," 07 2012.

[13] D. S. Berger, P. Gland, S. Singla, and F. Ciucu, "Exact analysis of ttl cache networks," *Performance Evaluation*, vol. 92, pp. 79–91, 2014.

[14] T. Wong and J. Wilkes, "My cache or yours? making storage more exclusive," in *2002 USENIX Annual Technical Conference (USENIX ATC 02)*. Monterey, CA: USENIX Association, Jun. 2002. [Online]. Available: https://www.usenix.org/conference/2002-usenix-annual-technical-conference/my-cache-or-yours-making-storage-more-exclusive

[15] A. R. Karlin, M. S. Manasse, L. Rudolph, and D. D. Sleator, "Competitive snoopy caching," *Algorithmica*, vol. 3, no. 1, pp. 79–119, Nov. 1988. [Online]. Available: https://doi.org/10.1007/BF01762111

[16] K. Gurushankar, N. G. Singer, and B. Subercaseaux, "Latency guarantees for caching with delayed hits," *Proceedings of IEEE INFOCOM*, 2025.

[17] H. Tan, Y. Wang, C. Zhang, G. Li, H. Du, Z. Han, S. H.-C. Jiang, and X.-Y. Li, "Asymptotically tight approximation for online file caching with delayed hits and bypassing," *IEEE Transactions on Networking*, vol. 33, no. 4, pp. 1886–1899, 2025.

[18] D. D. Sleator and R. E. Tarjan, "Amortized efficiency of list update and paging rules," *Commun. ACM*, vol. 28, no. 2, p. 202–208, Feb. 1985. [Online]. Available: https://doi.org/10.1145/2786.2793

[19] A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, and N. E. Young, "Competitive paging algorithms," *Journal of Algorithms*, vol. 12, no. 4, pp. 685–699, 1991. [Online]. Available: https://www.sciencedirect.com/science/article/pii/019667749190041V

[20] N. Buchbinder and J. S. Naor, "The design of competitive online algorithms via a primal—dual approach," *Foundations and Trends in Theoretical Computer Science*, vol. 3, no. 2-3, pp. 93–263, 05 2009. [Online]. Available: https://doi.org/10.1561/0400000024

[21] A. Adamaszek, A. Czumaj, M. Englert, and H. Räcke, "An o(log k)-competitive algorithm for generalized caching," *ACM Trans. Algorithms*, vol. 15, no. 1, Nov. 2018. [Online]. Available: https://doi.org/10.1145/3280826